ClearURL: Explainable AI-Based Phishing URL Detection System

Iteration 2 Run Sheet and Demo Guide

FIT5238 Group 15 | August, 2026

Public Deployment

The Iteration 2 system is deployed to Alibaba Cloud ECS and can be opened directly at:

http://120.26.118.110/

No local installation is required for the online demonstration. The hosted frontend communicates with the deployed backend, trained model artifact, and project databases.

Submission Entry Point

The links below provide direct access to the deployed system and supporting evidence.

Moodle upload	This PDF file
Public system	Open the deployed ClearURL system
GitLab repository	Open the GitLab repository
Live Trello board	Open the live Trello board
Shared folder	Open the shared Google Drive folder

Contents

1 Part I: Run and Reproduce the System **2**

1.1 Option A: Open the Deployed System . . . 2
1.2 Option B: Reproduce the System Locally . . . 2
1.3 Clone the Repository . . . 2
1.4 Start the Backend . . . 3
1.5 Start the Frontend . . . 3
1.6 Validate the Reproduction . . . 3
1.7 Local Recovery Checks . . . 3

2 Part II: Demonstration **3**

2.1 Demonstration Accounts and CAPTCHA . . . 3
2.2 Employee Analysis and Explanation . . . 4
2.3 Employee Feedback . . . 4
2.4 Administrator Case Resolution and Policy Override . . . 4
2.5 Administrator Monitoring . . . 5
2.6 Password Recovery and Registration . . . 5
2.7 Data Persistence Demonstration . . . 6
2.8 Demonstration Checklist . . . 6

3 Supporting Evidence **6**

Iteration 2 Evidence Map

Assessment requirement	Primary evidence	Location
Working system and demo	Deployed employee and administrator workflows	Public deployment
Code repository	Source, model artifact, API routes, tests, README	GitLab main branch
Technical documentation	Architecture, AI integration, API contract, persistence, deployment	`docs/Technical Documentation.pdf`
Planning and backlog	Iteration 2 stories, task cards, sprint status	Live Trello board and shared folder
Continuous testing	Automated tests, model test results, security and policy records	`docs/TEST_RECORD_*.md` and test reports
Review and adaptation	Employee feedback, administrator decisions, policy and retraining-data workflow	Deployed Admin console and `clearurl-feedback.sqlite3`

1 Part I: Run and Reproduce the System

This part provides two ways to reproduce ClearURL. The preferred method is the hosted system. The local method is a reproducible development and demonstration backup.

1.1 Option A: Open the Deployed System

Open the deployed system:

http://120.26.118.110/

No local installation is required. The hosted system includes the React frontend, Starlette backend, trained XGBoost artifact, and separate SQLite data stores for authentication, scans, feedback cases, and model records.

The system attempts live webpage retrieval before feature extraction. External network access, DNS, remote website controls, or server location can limit retrieval. In these cases, ClearURL displays a verification status such as `URL-only analysis` or `Destination could not be verified`; inaccessible content alone is not presented as proof of phishing.

1.2 Option B: Reproduce the System Locally

The local environment requires:

- Git
- Node.js `>= 22`
- Python `3.10+`; a virtual environment or Conda environment is recommended
- the repository files under `artifacts/` and `data/`

1.3 Clone the Repository

```
git clone https://git.infotech.monash.edu/fit5238-group15/url-detection.git
cd url-detection
git checkout main
git pull --ff-only origin main
```

The model artifact must exist at `artifacts/phishing_detection_model.joblib`. The data directory must be writable because the application creates sessions, prediction logs, feedback cases, and model records during use.

1.4 Start the Backend

From the project root, activate the environment, install dependencies, and start the Starlette API:

```
python -m pip install -r requirements-backend.txt
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

The backend should be available at `http://127.0.0.1:8000`. A quick health check is to request a protected route. An authentication-required response confirms that the backend is running:

```
curl http://127.0.0.1:8000/api/auth/session
```

1.5 Start the Frontend

Open a second terminal in the project root:

```
npm install
npm run dev
```

The Vite development server is normally available at `http://127.0.0.1:5173`. During local development, Vite proxies `/api/` requests to the backend on port `8000`.

1.6 Validate the Reproduction

Run backend regression tests and the production frontend build:

```
pytest -q
npm run build
```

Expected result: the backend tests pass and the frontend build completes without TypeScript or Vite errors. The test suite covers preprocessing, authentication and CAPTCHA, password recovery, analysis integration, official-domain handling, brand impersonation, policy overrides, administration APIs, and documentation availability.

1.7 Local Recovery Checks

If the backend does not start, confirm the Python environment, dependency installation, `data/` directory, and model artifact. If analysis fails in the browser, confirm that port `8000` is running and that the browser has an authenticated session. If the admin console cannot load, verify that the signed-in account has the `admin` role.

2 Part II: Demonstration

2.1 Demonstration Accounts and CAPTCHA

The seeded local demonstration accounts are:

- Employee: `employee@clearurl.local` / `employee123`
- Admin: `admin@clearurl.local` / `admin123`

Before signing in, enter the graphical CAPTCHA code displayed in the authentication card. The CAPTCHA image can be refreshed using `New image`. The same verification mechanism is used for registration, password changes, and password resets.

2.2 Employee Analysis and Explanation

1. Open the deployed system or the local frontend.
2. Select `Employee`, enter the employee credentials, enter the displayed CAPTCHA, and select `Sign in`.
3. In the URL field, enter `github.com/login` without a scheme.
4. Select `Analyse URL`.

Expected result:

- the input is accepted and normalised with `https://`
- the result displays the normalised URL and a `Safe`, `Suspicious`, or `High Risk` category
- the result displays a verification status explaining whether webpage evidence was retrieved
- the result displays URL-specific reasons and recommended actions
- the employee view does not display a numeric score; numeric risk and safety values remain in the API and administrative records

For brand-impersonation demonstration, use a controlled example such as `https://go0gle.com`. The explanation should identify the relevant domain identity evidence when detected, such as similarity to a monitored official domain or a digit-to-letter substitution. A live classification may vary if the deployment cannot retrieve the target webpage; the verification status provides that context.

2.3 Employee Feedback

1. After an analysis result, select `Send for review`.
2. Confirm the current system category shown by the form.
3. Select either `Confirm and report risk` or `Request a result review`.
4. Select an assessment and structured reason; optionally add a short note.
5. Select `Submit review`.

Expected result:

- the report is stored with the request ID, normalised URL, review type, employee assessment, reason code, note, and timestamp
- repeated reports for the same normalised URL are aggregated into one reviewable case
- the newly submitted feedback is available in the Admin `Review cases` view

2.4 Administrator Case Resolution and Policy Override

1. Sign out and sign in with the Admin demonstration account and CAPTCHA.
2. Select `Review cases` in the left navigation.

3. Open an available case using `Process case`.
4. Choose an `Allowlist` or `Blocklist` decision, enter a mandatory reason, and optionally attach screenshot evidence.
5. For an allowlist decision, choose a time-limited period or permanent status.
6. Continue to the confirmation step and apply the decision.

Expected result:

- the case status becomes `Allowlisted` or `Blocklisted`
- the decision reason, administrator identity, timestamp, expiry, and optional evidence metadata are retained
- the entry becomes visible in `Policy lists`
- a later scan of the same exact normalised URL uses the reviewed policy before webpage retrieval and model inference
- the returned verification status identifies an `ADMIN_ALLOWLIST_MATCH` or `ADMIN_BLOCKLIST_MATCH`

Use a non-production test URL for this demonstration. Policy decisions persist in the feedback database and should not be created for a real URL unless the team intends to retain the operational decision.

2.5 Administrator Monitoring

Use the Admin sidebar to demonstrate:

- **Overview**: total scans, open cases, reports, response time, risk distribution, and recorded model comparison
- **Scan records**: all logged analyses, with filter controls and pagination of forty records per page
- **Brand visualiser**: monitored-brand similarity, observed and target domains, character substitutions, techniques, and contextual risk information
- **Policy lists**: searchable allowlist and blocklist entries, expiry status, evidence metadata, and entry removal
- **Model monitoring**: held-out metrics separated from live operations and policy counts
- **Account management**: registered account details and confirmation-based suspension of active Employee accounts without deleting their history

2.6 Password Recovery and Registration

To demonstrate account security without altering a shared account, create a temporary test account:

1. Select `Register`, choose a role, enter a name, unique email, password, confirmation password, and CAPTCHA.
2. Submit registration and save the one-time recovery code displayed in the modal.
3. Sign out and use `Forgot password?`.
4. Enter the test-account email, saved recovery code, new password, and CAPTCHA.

Expected result: registration creates an active account and returns a session; password reset accepts only the matching recovery code and a valid CAPTCHA. The recovery code is shown once and stored server-side only as a hash.

2.7 Data Persistence Demonstration

ClearURL records operational data without automatically adding every analysed URL to the model training dataset:

- analyses are stored in `data/prediction_logs.sqlite3`
- review cases, administrator decisions, policy entries, and feedback metadata are stored in `data/clearurl-feedback.sqlite3`
- users, sessions, password hashes, recovery-code hashes, and account status are stored in `data/auth.sqlite3`
- training and evaluation records are stored in `data/model_records.sqlite3`

Reviewed feedback can be exported through the Admin feedback-dataset API for later retraining consideration. It does not automatically change the trained model.

2.8 Demonstration Checklist

1. the public URL or local frontend is reachable
2. CAPTCHA-protected Employee login works
3. a domain-only input is normalised and analysed
4. the result contains category, verification status, explanations, and recommendations
5. an employee review can be submitted
6. CAPTCHA-protected Admin login works
7. the new scan and review case are visible in the Admin console
8. a reviewed policy decision can be applied and displayed
9. model monitoring, brand visualiser, policy list, and account-management views load
10. backend tests and frontend production build have been checked

3 Supporting Evidence

The following repository files provide supporting evidence for this run sheet:

- `README.md`
- `docs/Technical Documentation.pdf`
- `docs/ITERATION2_USER_STORIES.md`
- `docs/USER_STORY_TASK_CARDS.md`
- `docs/MODEL_TEST_RESULTS.md`
- `docs/TEST_RECORD_6.md` through `docs/TEST_RECORD_9.md`
- `docs/TEST_RECORD_10_FINAL.md`
- `docs/GITLAB_UPLOAD_PLAN.md`